প্রহরী

Smart Fire & Gas Leak Response System

প্রহরী (Prohori) is Bangla for “guard” or “watchman”

Gas and flame detection with automatic door unlocking, sprinkler and live monitoring

Nabiha Nowrin · Department of EEE, University of Chittagong · Personal project, October 2026

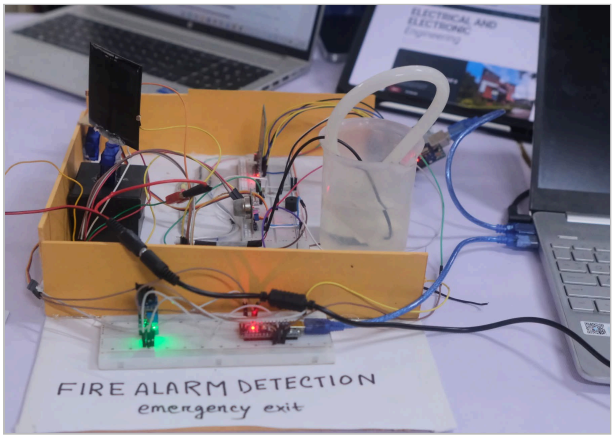


Figure 1. The project on display at the stall, October 2026.

Project type	Personal project, designed and built alone
Built	October 2026
Controllers	Arduino Uno (Zone A), Arduino Nano (Zone B)
Sensors	MQ-2 gas/smoke sensor, IR flame sensor, RC522 RFID reader
Outputs	Buzzer, water-pump sprinkler, servo door, 3-LED exit arrows
Monitoring	Python dashboard on a laptop over USB serial
Exhibited	CUCSU 1st National Research Festival 2026; Chittagong University Innovation Fair 2026

1 Overview

1.1 Problem

> **People are rarely trapped in a fire or a gas leak just because a door is locked. They are trapped because nobody noticed the danger fast enough to open it.**
>
> *Prohori* is a working model of a building that keeps that watch itself. It detects gas or flame, opens the exit door, starts a sprinkler, sounds an alarm, lights exit arrows, and reports which registered people are still inside.

1.2 What the system does

- **Zone A, kitchen (Arduino Uno):** an MQ-2 sensor watches for LPG and smoke. On danger it sounds the buzzer, runs the water-pump sprinkler, lights the exit arrows and opens the door.
- **Zone B, living room (Arduino Nano):** an IR flame sensor watches for fire. On fire it opens the door.
- **RFID door:** in normal mode the door opens only for a registered card. Each tap marks the person as inside or outside, so the system knows who is in the building.
- **Dashboard:** a Python program on a laptop shows both zones live and keeps an event log.

2 Hardware

2.1 Components

Component	Qty	Role
Arduino Uno	1	Zone A controller: gas sensing, RFID, pump, buzzer, exit LEDs
Arduino Nano	1	Zone B controller: flame sensing and the door servo
MQ-2 gas sensor module	1	Detects LPG and smoke (analog output)
IR flame sensor module	1	Detects an open flame (digital output)
RC522 RFID reader + card	1	Door access and the list of people inside
Servo motor	1	Opens and closes the door
Buzzer	1	Alarm sound
Small DC water pump	1	Sprinkler
IRFZ44N N-channel MOSFET	1	Switches the pump from an Arduino pin
Resistors: 220 Ω, 10 kΩ	1 each	Gate resistor and gate pull-down
Flyback diode	1	Protects the MOSFET from the pump's voltage spike
3.7 V single-cell LiPo	1	Separate supply for the pump
3-LED module (red, yellow, green)	1	Exit arrows
Breadboard, jumper wires, laptop	–	Wiring, power rails, USB power and monitoring

2.2 Pin connections

Arduino Uno	Connected to
A0	MQ-2 AO
D10, D13, D11, D12, D9	RC522 SDA, SCK, MOSI, MISO, RST
3.3V	RC522 3.3V
D5	Buzzer +
D3	220 Ω → MOSFET gate
D2, A1, A3	Exit LEDs red, yellow, green
D8	Nano D8 (door signal)
5V, GND	Breadboard rails, Nano GND

Arduino Nano	Connected to
D7	Flame sensor DO
D6	Servo signal
D8	Uno D8 (door signal)
5V	Flame sensor VCC, servo +
GND	Flame sensor, servo, Uno GND

Pins D0 and D1 are left empty on both boards, because they carry the USB serial link to the laptop.

2.3 Wiring diagram

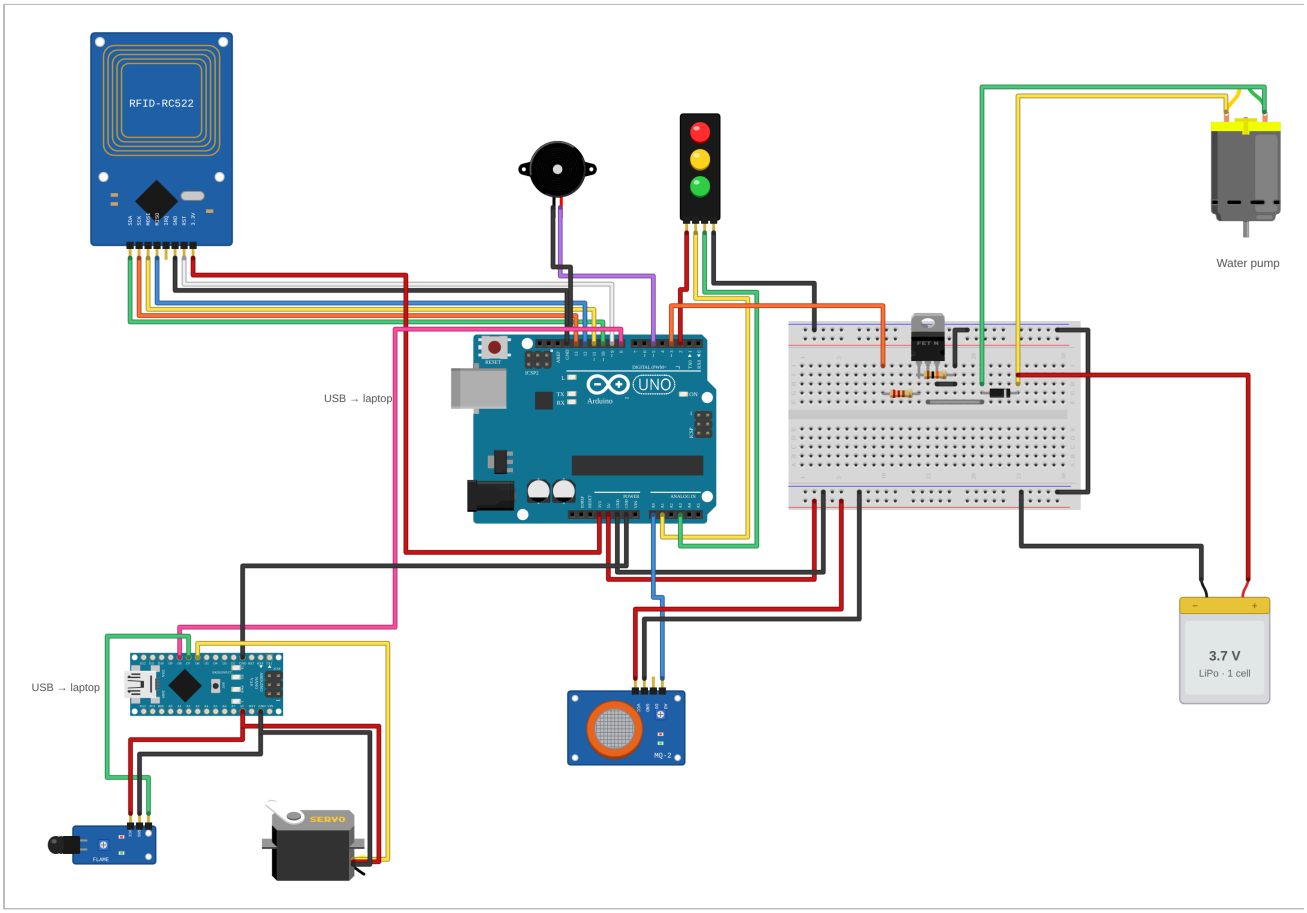


Figure 2. Wiring diagram of the final build. Part artwork for the Arduino boards, servo, buzzer, MOSFET, resistors, diode, motor (standing in for the pump) and breadboard is from the Fritzing parts library (CC BY-SA 3.0); the RC522, MQ-2, flame sensor, LED module and battery are drawn to match.

2.4 Design notes

- **Two boards, one ground.** Each board is powered by its own USB cable. A ground wire joins them, so the Uno's D8 signal means the same voltage on both boards.
- **Pump driver.** The pump draws more current than an Arduino pin can supply, so the pin drives the gate of an IRFZ44N MOSFET, which switches the pump's battery current. The 220 Ω resistor limits the gate current. The 10 kΩ pull-down keeps the pump off while the Uno is starting up.
- **Flyback diode.** When the pump switches off, its coil produces a voltage spike. The diode across the pump (stripe on the battery + side) gives that spike a safe path, so it does not damage the MOSFET.
- **RC522 on 3.3 V.** The reader is a 3.3 V device, so it is powered from the Uno's 3.3V pin, not 5V.

3 How it works

3.1 Responses

Situation	Detected by	Response
Normal	–	Door locked; opens for 3 s for a registered card, which marks that person in or out
Unknown card	RC522	Door stays locked; “ACCESS DENIED” is logged with the card's ID
Gas or smoke	MQ-2 reading above 330	Buzzer on, pump sprinkler on, exit arrows run red → yellow → green, door opens, list of people inside is reported
Fire	IR flame sensor	Door opens; the alarm is held for 2 s after the flame disappears
Back to safe	Reading below 330 / no flame	Outputs switch off and the door closes

3.2 Zone A: Arduino Uno

Every 200 ms the Uno reads the MQ-2 analog output (0–1023). In clean air the reading was about 195, and the danger threshold is set at 330. Above it, the Uno switches on the buzzer and the pump, steps the three exit LEDs one after another so they look like a moving arrow, and sets D8 HIGH to tell the Nano to open the door. While the air is safe, it checks the RFID reader. A registered card switches that person between inside and outside, and opens the door for 3 seconds.

3.3 Zone B: Arduino Nano

The flame sensor's output goes LOW when it sees a flame. The Nano keeps the alarm on for 2 seconds after the last flame reading, so the door does not flicker open and shut. The door servo opens (180°) if there is a fire *or* if the Uno's D8 signal is HIGH, and closes (90°) otherwise.

3.4 Messages to the laptop

Both boards send one status line every 200 ms over USB at 9600 baud, plus an event line whenever something happens:

```
A|gas=196|alarm=0|door=0|fan=0|pump=0|lights=0|inside=nobody
B|flame=0|alarm=0|door=0|lights=0
EV|A|GAS ALERT - door open, sprinkler ON
EV|A|Still inside: Person 1
```

The `fan` field is always 0. An exhaust fan was planned but left out of the final build.

4 Monitoring dashboard

The dashboard is a Python program (pyserial and matplotlib) that runs on the laptop. It finds both boards on the USB ports by itself and reconnects if a cable is pulled out. For each zone it shows:

- the status (SAFE, WARNING or ALERT) and, for Zone A, a gas-level gauge with warning (260) and danger (330) marks;
- door, sprinkler and exit-light status, and the people inside according to the RFID log;
- a graph of the last 30 seconds of readings;
- an event log ("black box"), which is also saved to `blackbox.csv` with a timestamp for every event.

After an alarm lasts 3 seconds, the dashboard shows "Fire Service alerted (999)". **This call is simulated.** No call or message is actually sent.

The dashboard works over USB, not Wi-Fi, so it can only be viewed on the laptop. A Wi-Fi version for phones was planned, but the ESP32 board it needed burned out during the build.

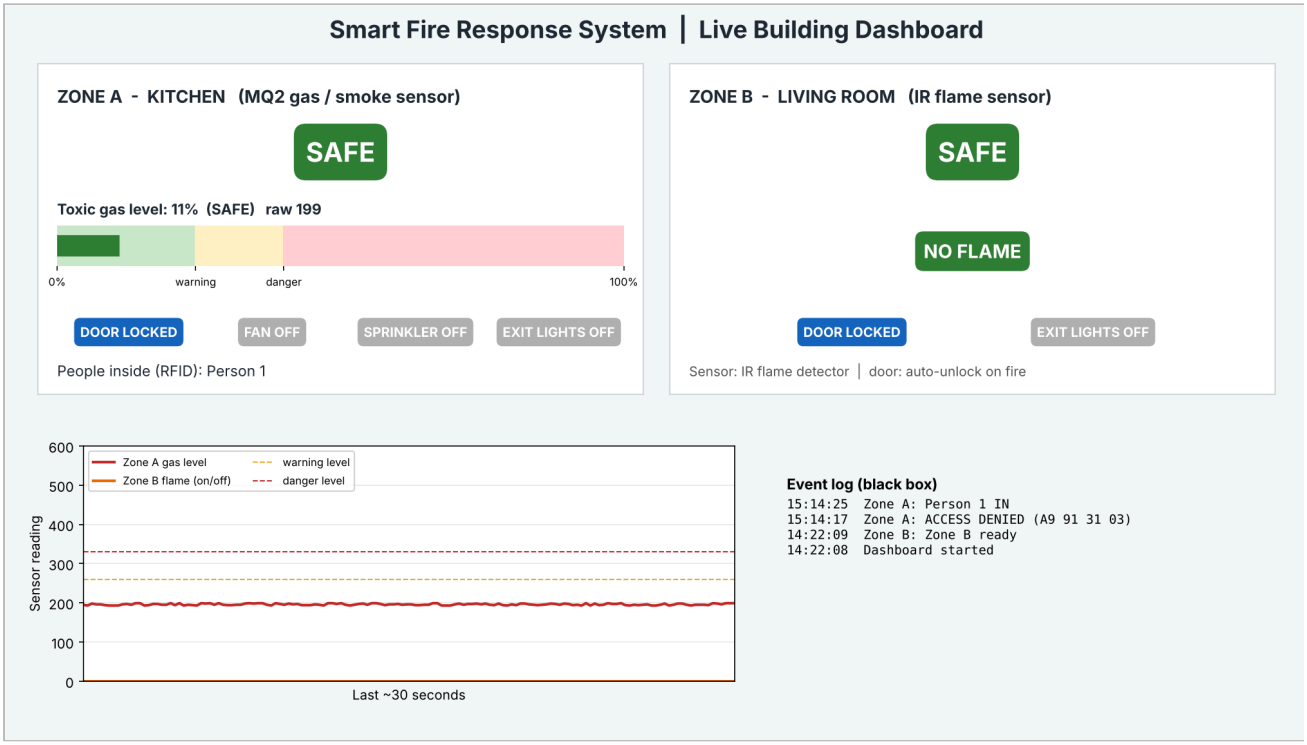


Figure 3. Dashboard with both zones safe.

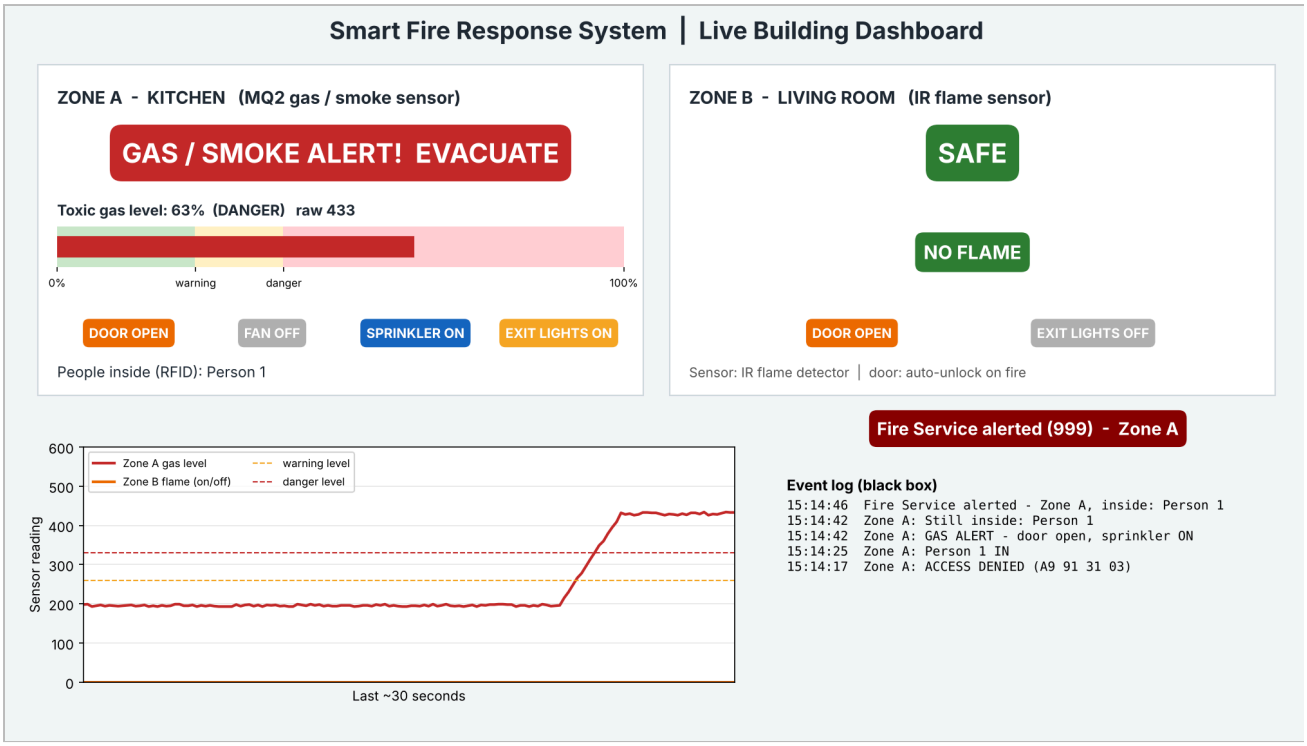


Figure 4. Dashboard during a gas alert in Zone A: door open, sprinkler and exit lights on, Fire Service alert (simulated).

Figures 3 and 4 were rendered from the project's own `dashboard.py` with sample sensor values, because the hardware is no longer assembled. The event-log lines shown are real entries from `blackbox.csv` (14 October 2026).

5 Test log

The dashboard's `blackbox.csv` recorded every event during testing and the exhibition on 14 October 2026, from 12:56 to 16:19. It records events only, not the sensor readings themselves.

Event	Count	Notes
Fire alerts (Zone B)	12	All 12 were followed by “Fire clear”
Gas alerts (Zone A)	9	Each one reported who was inside
Fire Service alerts (simulated)	5	3 for Zone B, 2 for Zone A
RFID card accepted	1	“Person 1 IN”
RFID card refused	1	Unregistered card A9 91 31 03
Dashboard starts	8	Restarts while setting up and uploading code
USB disconnects	4	Cables unplugged during setup

One gas alert from an earlier test build that morning (with the planned fan) is not counted.

Finding: at 16:18:54 the log shows four gas alerts and four “air clear” events within the same second. The reading was hovering right at the 330 threshold, so the alarm switched on and off repeatedly. A fix is in Section 7.

6 Exhibitions and recognition

Event	Date	Venue	Result
CUCSU 1st National Research Festival 2026	13 Oct 2026	Shaheed Minar & Buddhijibi Chattar, University of Chittagong	Best Stall, IT Faculty (shared stall)
Chittagong University Innovation Fair 2026 (Project Showcasing)	14 Oct 2026	CUCSU Building, University of Chittagong	Participation

The project was designed and built by me alone. At the Innovation Fair it was entered under the four-member team “Thinking Too Much”, because that segment required teams of three to five. At the Research Festival, an earlier version ran on the Uno alone. The Nano, flame sensor and door link were added for the Innovation Fair.

7 Limitations and future work

Limitation	Improvement
The alarm flickers when the gas reading sits at the threshold	Hysteresis: switch on above 330, switch off only below about 300, and average several readings
Monitoring works only on the laptop (USB)	An ESP32 with Wi-Fi and a web dashboard, so the system can be checked on a phone
The Fire Service call is simulated	A GSM module to send a real SMS or call
The MQ-2 threshold comes from my own test readings, not a calibration in ppm, and the sensor cannot tell LPG from smoke	Calibrate against a known gas concentration; add a second sensor type
The IR flame sensor has a short range and can react to sunlight	Combine it with a smoke or temperature reading before raising the alarm
A servo holds its last position if power fails	In a real building, use a lock that opens when power is lost (fail-safe)
The IRFZ44N is not a logic-level MOSFET, so a 5 V gate only partly turns it on	Use a logic-level MOSFET such as the IRLZ44N
Only events are logged, not readings	Save the gas readings too, so alarms can be analysed afterwards

Appendix A Arduino Uno code (Zone A)

```
#include <SPI.h>
#include <MFRC522.h>

const int mq2Pin = A0;
const int pumpPin = 3;     // MOSFET gate (through 220 ohm)
const int buzzerPin = 5;
const int linkPin = 8;     // to Nano D8: HIGH = open door
const int arrowLeds[] = {2, A1, A3};   // red, yellow, green
const int arrowCount = 3;
int arrowStep = 0;

int gasDanger = 330;

MFRC522 rfid(10, 9);

String cardUID[] = {"14 59 37 03"};
String cardName[] = {"Person 1"};
bool inside[] = {false};
const int cardCount = 1;

bool alarmOn = false;
bool doorOpen = false;

void setup() {
  Serial.begin(9600);
  pinMode(pumpPin, OUTPUT);
  digitalWrite(pumpPin, LOW);
  pinMode(buzzerPin, OUTPUT);
  pinMode(linkPin, OUTPUT);
  digitalWrite(linkPin, LOW);
  for (int i = 0; i < arrowCount; i++) pinMode(arrowLeds[i], OUTPUT);
  arrowsOff();
  SPI.begin();
  rfid.PCD_Init();
  Serial.println("EV|A|Zone A ready");
}

void loop() {
  int gas = analogRead(mq2Pin);

  if (gas > gasDanger) {
    if (!alarmOn) {
      alarmOn = true;
      Serial.println("EV|A|GAS ALERT - door open, sprinkler ON");
      Serial.println("EV|A|Still inside: " + insideList());
    }
    digitalWrite(buzzerPin, HIGH);
    digitalWrite(pumpPin, HIGH);
    setDoor(true);
    runArrows();
  } else {
    if (alarmOn) {
      alarmOn = false;
      Serial.println("EV|A|Air clear - door closed");
    }
    digitalWrite(buzzerPin, LOW);
    digitalWrite(pumpPin, LOW);
    setDoor(false);
    arrowsOff();
    checkCard(gas);
  }

  sendStatus(gas);
  delay(200);
}

void setDoor(bool open) {
  doorOpen = open;
  digitalWrite(linkPin, open ? HIGH : LOW);
}
```

```
void runArrows() {
  arrowsOff();
  digitalWrite(arrowLeds[arrowStep], HIGH);
  arrowStep = (arrowStep + 1) % arrowCount;
}

void arrowsOff() {
  for (int i = 0; i < arrowCount; i++) digitalWrite(arrowLeds[i], LOW);
}

void sendStatus(int gas) {
  Serial.print("A|gas=");   Serial.print(gas);
  Serial.print("|alarm=");  Serial.print(alarmOn ? 1 : 0);
  Serial.print("|door=");   Serial.print(doorOpen ? 1 : 0);
  Serial.print("|fan=0|pump=");
  Serial.print(alarmOn ? 1 : 0);
  Serial.print("|lights=");
  Serial.print(alarmOn ? 1 : 0);
  Serial.print("|inside="); Serial.println(insideList());
}

void checkCard(int gas) {
  if (!rfid.PICC_IsNewCardPresent()) return;
  if (!rfid.PICC_ReadCardSerial()) return;

  String uid = readUID();
  int person = findCard(uid);
  if (person == -1) {
    Serial.println("EV|A|ACCESS DENIED (" + uid + ")");
  } else {
    inside[person] = !inside[person];
    Serial.println("EV|A|" + cardName[person] + (inside[person] ? " IN" : " OUT"));
    setDoor(true);
    sendStatus(gas);
    delay(3000);
    setDoor(false);
  }
  rfid.PICC_HaltA();
}

String readUID() {
  String uid = "";
  for (byte i = 0; i < rfid.uid.size; i++) {
    if (i > 0) uid += " ";
    if (rfid.uid.uidByte[i] < 0x10) uid += "0";
    uid += String(rfid.uid.uidByte[i], HEX);
  }
  uid.toUpperCase();
  return uid;
}

int findCard(String uid) {
  for (int i = 0; i < cardCount; i++) {
    if (uid == cardUID[i]) return i;
  }
  return -1;
}

String insideList() {
  String list = "";
  for (int i = 0; i < cardCount; i++) {
    if (inside[i]) {
      if (list != "") list += ", ";
      list += cardName[i];
    }
  }
  if (list == "") list = "nobody";
  return list;
}
```

Appendix B Arduino Nano code (Zone B)

```
#include <Servo.h>

const int flamePin = 7;
const int servoPin = 6;
const int linkPin = 8;     // from Uno D8

const int FLAME_DETECTED = LOW;
const int closedAngle = 90;
const int openAngle = 180;

Servo door;
bool alarmOn = false;
bool doorOpen = false;
unsigned long lastFlame = 0;

void setup() {
  Serial.begin(9600);
  pinMode(flamePin, INPUT);
  pinMode(linkPin, INPUT);
  door.attach(servoPin);
  door.write(closedAngle);
  Serial.println("EV|B|Zone B ready");
}

void loop() {
  bool flame = (digitalRead(flamePin) == FLAME_DETECTED);
  if (flame) lastFlame = millis();

  bool fireAlarm = flame || (lastFlame > 0 && millis() - lastFlame < 2000);
  bool unoSaysOpen = (digitalRead(linkPin) == HIGH);

  if (fireAlarm && !alarmOn) Serial.println("EV|B|FIRE ALERT - door open");
  if (!fireAlarm && alarmOn) Serial.println("EV|B|Fire clear");
  alarmOn = fireAlarm;

  bool openNow = fireAlarm || unoSaysOpen;
  if (openNow != doorOpen) {
    doorOpen = openNow;
    door.write(doorOpen ? openAngle : closedAngle);
  }

  Serial.print("B|flame=");  Serial.print(flame ? 1 : 0);
  Serial.print("|alarm=");   Serial.print(alarmOn ? 1 : 0);
  Serial.print("|door=");    Serial.print(doorOpen ? 1 : 0);
  Serial.println("|lights=0");
  delay(200);
}
```

Appendix C Dashboard (dashboard.py)

```
import csv
import math
import time
from collections import deque
from datetime import datetime

import serial
from serial.tools import list_ports
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
from matplotlib.patches import Rectangle

# ---------------- Settings ----------------
BAUD = 9600
GAS_WARN = 260        # gas reading where the gauge turns yellow
GAS_DANGER = 330      # must match gasDanger in ZoneA_Uno.ino
GAS_MIN, GAS_MAX = 150, 600   # range shown on the toxic-level gauge
POINTS = 150          # about 30 s of graph history
CALL_DELAY = 3        # seconds of "Calling..." before "alerted"
NO_DATA_SECS = 5
PORT_WORDS = ("arduino", "ch340", "ch341", "usb-serial", "usb serial", "wch", "ch9102", "cp210")

GREEN, AMBER, RED, GRAY = "#2e7d32", "#f9a825", "#c62828", "#757575"
BLUE, ORANGE, OFF = "#1565c0", "#ef6c00", "#b0b0b0"

# ---------------- State ----------------
links = {}            # port name -> serial object
last_scan = [0.0]
zones = {
    "A": {"seen": 0.0, "gas": None, "alarm": 0, "door": 0, "fan": 0, "pump": 0,
          "lights": 0, "inside": "nobody", "alarm_start": 0.0, "called": False},
    "B": {"seen": 0.0, "flame": 0, "alarm": 0, "door": 0, "lights": 0,
          "alarm_start": 0.0, "called": False},
}
gas_hist = deque([math.nan] * POINTS, maxlen=POINTS)
flame_hist = deque([math.nan] * POINTS, maxlen=POINTS)
events = deque(maxlen=9)

log_file = open("blackbox.csv", "a", newline="")
log = csv.writer(log_file)


def add_event(text):
    events.appendleft(f"{datetime.now():%H:%M:%S}  {text[:52]}")
    log.writerow([datetime.now().isoformat(timespec="seconds"), text])
    log_file.flush()


# ---------------- Serial: find and keep both boards connected ----------------
def scan_ports():
    last_scan[0] = time.time()
    for p in list_ports.comports():
        if p.device in links:
            continue
        text = f"{p.description} {p.manufacturer}".lower()
        if any(w in text for w in PORT_WORDS):
            try:
                links[p.device] = serial.Serial(p.device, BAUD, timeout=0.05)
                print(f"Connected: {p.device} ({p.description})")
            except Exception as err:
                print(f"Could not open {p.device}: {err}")


def drop(port):
    try:
        links[port].close()
    except Exception:
        pass
    links.pop(port, None)
    add_event(f"USB lost on {port}")
```

```
def parse(line):
    if line.startswith("EV|"):
        parts = line.split("|", 2)
        if len(parts) == 3:
            add_event(f"Zone {parts[1]}: {parts[2]}")
        return
    if line[:2] not in ("A|", "B|"):
        return
    zone = zones[line[0]]
    was_alarm = zone["alarm"]
    for item in line[2:].split("|"):
        if "=" not in item:
            continue
        key, val = item.split("=", 1)
        if key == "inside":
            zone["inside"] = val.strip() or "nobody"
        else:
            try:
                zone[key] = int(val)
            except ValueError:
                pass
    zone["seen"] = time.time()
    if zone["alarm"] and not was_alarm:
        zone["alarm_start"] = time.time()
        zone["called"] = False


def read_all():
    if time.time() - last_scan[0] > 2:
        scan_ports()
    for port, ser in list(links.items()):
        try:
            while ser.in_waiting:
                raw = ser.readline().decode(errors="ignore").strip()
                if raw:
                    parse(raw)
        except Exception:
            drop(port)


def online(z):
    return time.time() - zones[z]["seen"] < NO_DATA_SECS


# ---------------- Window layout ----------------
fig = plt.figure(figsize=(13.5, 7.6), facecolor="#f4f6f8")
fig.canvas.manager.set_window_title("Smart Fire Response - Live Dashboard")
fig.text(0.5, 0.955, "Smart Fire Response System  |  Live Building Dashboard",
         ha="center", fontsize=18, weight="bold", color="#1f2a36")


def panel(x0, w, title):
    fig.patches.append(Rectangle((x0, 0.47), w, 0.45, transform=fig.transFigure,
                                 facecolor="white", edgecolor="#d0d7de", lw=1, zorder=-5))
    fig.text(x0 + 0.015, 0.89, title, fontsize=13, weight="bold", color="#1f2a36", va="top")


def chip(x, y, size=10):
    return fig.text(x, y, "", ha="center", va="center", fontsize=size, weight="bold", color="white",
                    bbox=dict(boxstyle="round,pad=0.45", facecolor=OFF, edgecolor="none"))


def set_chip(c, text, color):
    c.set_text(text)
    c.get_bbox_patch().set_facecolor(color)


panel(0.025, 0.465, "ZONE A  -  KITCHEN   (MQ2 gas / smoke sensor)")
panel(0.51, 0.465, "ZONE B  -  LIVING ROOM   (IR flame sensor)")

# Zone A
a_status = chip(0.2575, 0.80, 20)
a_level = fig.text(0.04, 0.715, "", fontsize=11, weight="bold", color="#1f2a36")
gauge = fig.add_axes([0.04, 0.645, 0.435, 0.055], zorder=5)
```

```
def pct(v):
    return max(0.0, min(100.0, (v - GAS_MIN) / (GAS_MAX - GAS_MIN) * 100))


gauge.add_patch(Rectangle((0, 0), pct(GAS_WARN), 1, color="#c8e6c9"))
gauge.add_patch(Rectangle((pct(GAS_WARN), 0), pct(GAS_DANGER) - pct(GAS_WARN), 1, color="#fff3c4"))
gauge.add_patch(Rectangle((pct(GAS_DANGER), 0), 100 - pct(GAS_DANGER), 1, color="#ffcdd2"))
gauge_bar = gauge.add_patch(Rectangle((0, 0.25), 0, 0.5, color=GREEN))
gauge.set_xlim(0, 100)
gauge.set_ylim(0, 1)
gauge.set_yticks([])
gauge.set_xticks([0, pct(GAS_WARN), pct(GAS_DANGER), 100])
gauge.set_xticklabels(["0%", "warning", "danger", "100%"], fontsize=8)
for s in gauge.spines.values():
    s.set_visible(False)
a_door, a_fan, a_pump, a_lights = (chip(x, 0.555) for x in (0.095, 0.205, 0.315, 0.425))
a_inside = fig.text(0.04, 0.495, "", fontsize=11, color="#1f2a36")

# Zone B
b_status = chip(0.7425, 0.80, 20)
b_flame = chip(0.7425, 0.665, 14)
b_door, b_lights = (chip(x, 0.555) for x in (0.65, 0.835))
fig.text(0.525, 0.495, "Sensor: IR flame detector  |  door: auto-unlock on fire",
         fontsize=10, color="#555555")

# Graph
graph = fig.add_axes([0.06, 0.08, 0.50, 0.32])
line_gas, = graph.plot(range(POINTS), list(gas_hist), color=RED, lw=2, label="Zone A gas level")
line_flame, = graph.step(range(POINTS), list(flame_hist), color=ORANGE, lw=2, label="Zone B flame
(on/off)")
graph.axhline(GAS_WARN, color=AMBER, ls="--", lw=1, label="warning level")
graph.axhline(GAS_DANGER, color=RED, ls="--", lw=1, label="danger level")
graph.set_xlim(0, POINTS - 1)
graph.set_ylim(0, GAS_MAX)
graph.set_xticks([])
graph.set_xlabel("Last ~30 seconds")
graph.set_ylabel("Sensor reading")
graph.legend(loc="upper left", fontsize=8, ncol=2)
graph.grid(alpha=0.3)

# Emergency call + event log
call_box = chip(0.785, 0.425, 13)
log_ax = fig.add_axes([0.60, 0.04, 0.38, 0.29])
log_ax.axis("off")
log_ax.set_title("Event log (black box)", loc="left", fontsize=11, weight="bold")
log_text = log_ax.text(0, 1, "", va="top", fontsize=9.5, family="monospace")


# ---------------- Update loop ----------------
def update(_frame):
    read_all()
    za, zb = zones["A"], zones["B"]
    a_on, b_on = online("A"), online("B")

    # Zone A
    gas = za["gas"]
    if not a_on or gas is None:
        set_chip(a_status, "NO DATA - check Uno USB", GRAY)
        a_level.set_text("Toxic gas level: --")
        gauge_bar.set_width(0)
        gas_hist.append(math.nan)
    else:
        p = pct(gas)
        if za["alarm"]:
            set_chip(a_status, "GAS / SMOKE ALERT!  EVACUATE", RED)
            level, col = "DANGER", RED
        elif gas >= GAS_WARN:
            set_chip(a_status, "WARNING - gas rising", AMBER)
            level, col = "WARNING", AMBER
        else:
            set_chip(a_status, "SAFE", GREEN)
            level, col = "SAFE", GREEN
        a_level.set_text(f"Toxic gas level: {p:.0f}%  ({level})   raw {gas}")
```

```
        gauge_bar.set_width(p)
        gauge_bar.set_color(col)
        gas_hist.append(gas)
    set_chip(a_door, "DOOR OPEN" if za["door"] else "DOOR LOCKED", ORANGE if za["door"] else BLUE)
    set_chip(a_fan, "FAN ON" if za["fan"] else "FAN OFF", BLUE if za["fan"] else OFF)
    set_chip(a_pump, "SPRINKLER ON" if za["pump"] else "SPRINKLER OFF", BLUE if za["pump"] else OFF)
    set_chip(a_lights, "EXIT LIGHTS ON" if za["lights"] else "EXIT LIGHTS OFF", AMBER if za["lights"]
else OFF)
    a_inside.set_text(f"People inside (RFID): {za['inside']}")

    # Zone B
    if not b_on:
        set_chip(b_status, "NO DATA - check Nano USB", GRAY)
        set_chip(b_flame, "FLAME: --", OFF)
        flame_hist.append(math.nan)
    else:
        if zb["alarm"]:
            set_chip(b_status, "FIRE DETECTED!  EVACUATE", RED)
        else:
            set_chip(b_status, "SAFE", GREEN)
        set_chip(b_flame, "FLAME DETECTED" if zb["flame"] else "NO FLAME", RED if zb["flame"] else GREEN)
        flame_hist.append(GAS_MAX * 0.9 if zb["flame"] else 0)
    set_chip(b_door, "DOOR OPEN" if zb["door"] else "DOOR LOCKED", ORANGE if zb["door"] else BLUE)
    set_chip(b_lights, "EXIT LIGHTS ON" if zb["lights"] else "EXIT LIGHTS OFF", AMBER if zb["lights"]
else OFF)

    # Emergency call (simulated - no real call is made)
    active = [(k, z) for k, z in zones.items() if z["alarm"] and online(k)]
    if active:
        names = " + ".join(k for k, _ in active)
        waited = time.time() - min(z["alarm_start"] for _, z in active)
        if waited < CALL_DELAY:
            set_chip(call_box, f"Calling Fire Service (999){'.' * (int(waited * 2) % 4)}", ORANGE)
        else:
            set_chip(call_box, f"Fire Service alerted (999)  -  Zone {names}", "#8b0000")
            for k, z in active:
                if not z["called"]:
                    z["called"] = True
                    extra = f", inside: {zones['A']['inside']}" if k == "A" else ""
                    add_event(f"Fire Service alerted - Zone {k}{extra}")
    else:
        call_box.set_text("")
        call_box.get_bbox_patch().set_facecolor("none")

    line_gas.set_ydata(list(gas_hist))
    line_flame.set_ydata(list(flame_hist))
    log_text.set_text("\n".join(events))
    return ()


add_event("Dashboard started")
scan_ports()
animation = FuncAnimation(fig, update, interval=200, cache_frame_data=False)
plt.show()

for s in list(links.values()):
    try:
        s.close()
    except Exception:
        pass
log_file.close()
```